

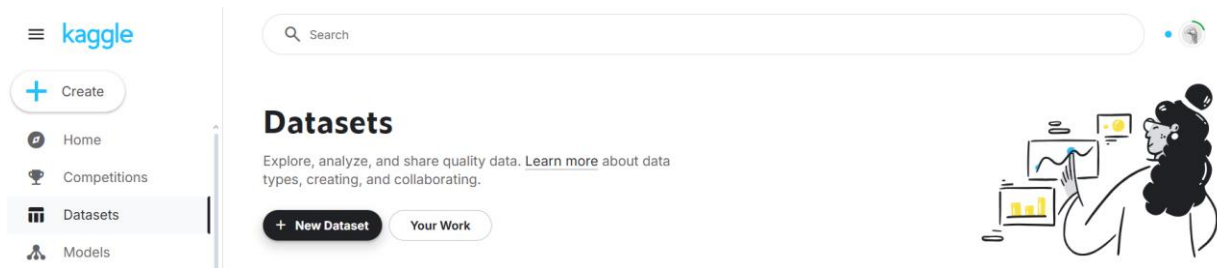
(فهرست مطالب)

1. پیاده سازی 3 مدل یادگیری **naïve bayes** و **Cart, C4.5**
 - نحوه اپلودکردن دیتاست در **kaggle**
2. رسم درخت های تصمیم گیری
3. مقایسه عملکرد
4. پیاده سازی مدل های **random forest** و **XGboost**
5. مقایسه این مدل ها
6. مقایسه با روش **k-fold** و نتیجه گیری نهایی

(پایاده سازی مدل های C4.5, Cart و naïve bayes)

(نحوه آپلود کردن دیتاست در kaggle)

با رفتن به بخش datasets در Kaggle و سپس با کلیک بر روی new dataset میتوان دیتاست مدنظر را آپلود کرد تا در مراحل بعدی از آن استفاده کنیم.



همچنین در بخش your work میتوان دیتاست های خود که پیشتر آپلود کرده اید را مشاهده کنید همانطور که مشاهده میکنید من یک دیتاست آپلود کرده ام و نام آن را decision tree گذاشته ام که آن در مراحل بعد استفاده کرده ام.

Your Work

This is the **private view** of your content. To see what others see, visit [Your Profile](#)

+ Create

Overview Collections Code Datasets Models Competitions Discussions Bookmarks

Your Datasets (2)

Search Your Work

All Filters Owner Privacy Type Size License Tags

Recently Updated

0 selected

☐  **decision tree**
ZAHRA ABR · Updated 25 days ago
Private · Usability 1.2 · 1 File (CSV) · 9 kB

0

...

(تعریف library و load کردن دیتاست)

Pandas: جهت load کردن دیتاست به کار میرود.

Sklearn: جهت جداسازی دیتاست به دو بخش train و test، جهت ایجاد decision tree و naïve bayes و همچنین random forest محاسبه معیارهای ارزیابی مدل ها به کار میرود.

Matplotlib: جهت visualization نمودارها و درخت تشکیل شده به کار میرود.

```
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt

# Load dataset from the Kaggle input directory
dataset = pd.read_csv('/kaggle/input/decision-tree/Dataset.csv')
```

سپس دیتاست را به دو بخش 20 درصد (برای test) و 80 درصد (برای train) تقسیم میکنیم و سپس target را از باقی feature ها جدا میکنیم.

```
# Split the data >> 20 percent test data and 80 percent train data
train_data, test_data = train_test_split(dataset, test_size=0.2, random_state=42)

# Separate features and target
X_train = train_data.drop(columns=['Outcome'])
y_train = train_data['Outcome']
X_test = test_data.drop(columns=['Outcome'])
y_test = test_data['Outcome']
```

سپس مدل ها را با داده های train پیاده سازی میکنیم. (من مشخص کرده ام که تنها تا سطح 5 درخت های دو مدل cart و c4.5 گسترش یابند.) مدل cart از gini index، مدل c4.5 از information gain و مدل naïve bayes برای قانون bayes استفاده میکنند.

```
# Model 1: CART
cart_model = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)
cart_model.fit(X_train, y_train)

# Model 2: C4.5 Approximation
c45_model = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)
c45_model.fit(X_train, y_train)

# Model 3: Naive Bayes
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
```

جهت به نمایش در آوردن درخت های تشکیل شده دو مدل cart و c4.5 :

```
# Plotting CART Tree
plt.figure(figsize=(15, 10))
plot_tree(cart_model, filled=True, feature_names=X_train.columns, class_names=["No Diabetes", "Diabetes"])
plt.title("CART Decision Tree")
plt.show()

# Plotting C4.5 Tree
plt.figure(figsize=(15, 10))
plot_tree(c45_model, filled=True, feature_names=X_train.columns, class_names=["No Diabetes", "Diabetes"])
plt.title("C4.5 Decision Tree")
plt.show()
```

سپس دو مدل random forest و XGboost را نیز با داده های train تشکیل میدهیم.

```
# Model 4: Random Forest
rf_model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf_model.fit(X_train, y_train)

# Model 5: XGBoost
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
```

سپس برای بدست آوردن معیار های ارزیابی هریک از مدل ها مدل ها را در یک دیکشنری قرار میدهیم تا سپس بتوانیم با کمک یک حلقه تک تک معیار ها را برای آنها محاسبه کنیم تا بتوانیم در نهایت عملکرد آنها را با هم مقایسه کنیم.

```
# Testing and calculating metrics
models = {
    "CART": cart_model,
    "C4.5": c45_model,
    "Naive Bayes": nb_model,
    "Random Forest": rf_model,
    "XGBoost": xgb_model,
}

for model_name, model in models.items():
    predictions = model.predict(X_test)
    print(f"\n{model_name} Model Metrics:")
    print(" Accuracy:", accuracy_score(y_test, predictions))
    print(" Precision:", precision_score(y_test, predictions))
    print(" Recall:", recall_score(y_test, predictions))
    print(" F1-Measure:", f1_score(y_test, predictions))
```

پس از آن با کمک رو 5-fold نیز اقدام به بدست آوردن accuracy تک تک مدل ها میکنیم تا بتوانیم درک کنیم نحوه محاسبه دقت مدل ها تا چه اندازه دارای اهمیت است.

```
# K-Fold Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
print("\nK-Fold Cross-Validation Results:")

for model_name, model in models.items():
    if model_name == "Naive Bayes": # Handle Naive Bayes separately due to probability scaling
        kfold_scores = cross_val_score(model, X_train, y_train, cv=kf, scoring='accuracy')
    else:
        kfold_scores = cross_val_score(model, X_train, y_train, cv=kf, scoring='accuracy')
    print(f"{model_name}: Mean Accuracy = {kfold_scores.mean():.4f}, Std Dev = {kfold_scores.std():.4f}")
```

پس از تمامی این مراحل خروجی کد بنده به شرح زیر است که در ادامه به بررسی آن و مقایسه پاسخ ها میپردازم:

CART Model Metrics:

Accuracy: 0.7922077922077922
Precision: 0.7446808510638298
Recall: 0.6363636363636364
F1-Measure: 0.6862745098039216

C4.5 Model Metrics:

Accuracy: 0.7857142857142857
Precision: 0.7037037037037037
Recall: 0.6909090909090909
F1-Measure: 0.6972477064220184

Naive Bayes Model Metrics:

Accuracy: 0.7662337662337663
Precision: 0.6610169491525424
Recall: 0.7090909090909091
F1-Measure: 0.6842105263157895

Random Forest Model Metrics:

Accuracy: 0.7662337662337663
Precision: 0.6862745098039216
Recall: 0.6363636363636364
F1-Measure: 0.660377358490566

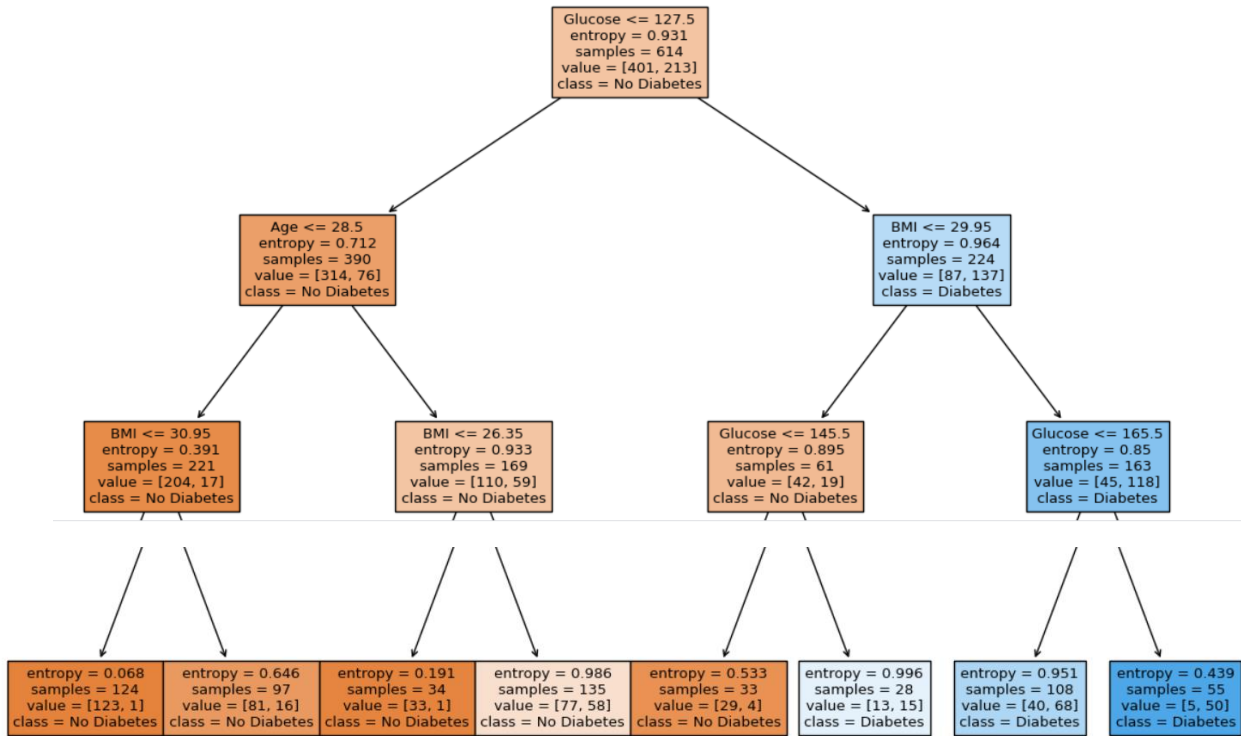
XGBoost Model Metrics:

Accuracy: 0.7077922077922078
Precision: 0.5806451612903226
Recall: 0.6545454545454545
F1-Measure: 0.6153846153846153

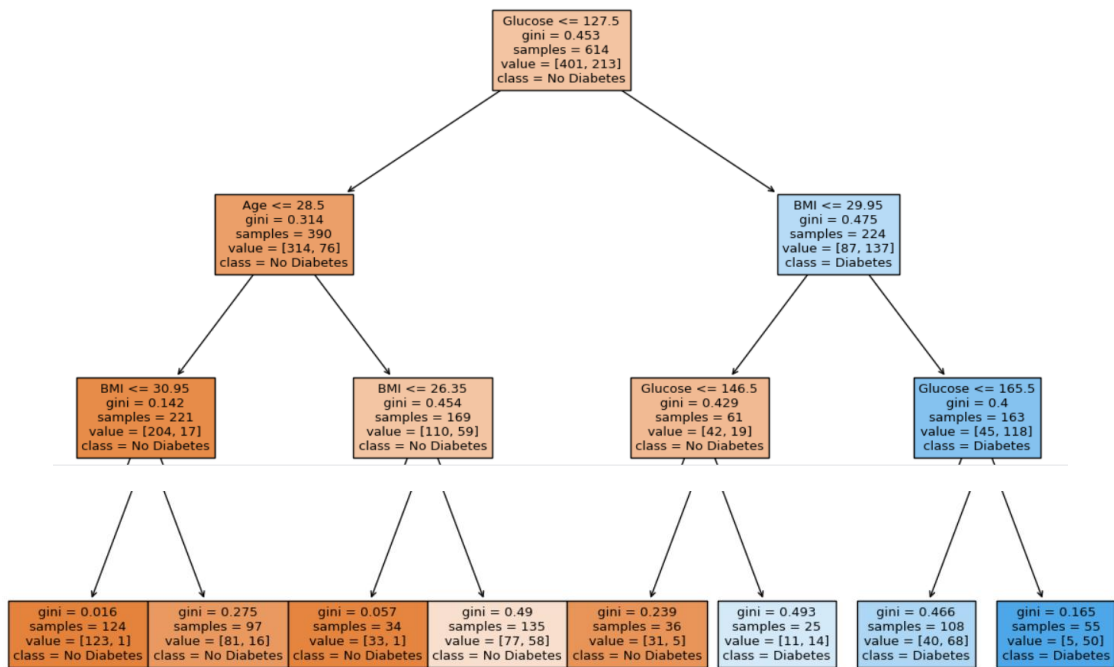
K-Fold Cross-Validation Results:

CART: Mean Accuracy = 0.7298, Std Dev = 0.0481
C4.5: Mean Accuracy = 0.7281, Std Dev = 0.0415
Naive Bayes: Mean Accuracy = 0.7363, Std Dev = 0.0357
Random Forest: Mean Accuracy = 0.7671, Std Dev = 0.0262
XGBoost: Mean Accuracy = 0.7444, Std Dev = 0.0288

C4.5 Decision Tree



CART Decision Tree



(نتایج مدل ها در قالب یک جدول)

	accuracy	precision	recall	F1-measure	5-fold accuracy
cart	79%	74%	63%	68%	72%
C4.5	78%	70%	69%	69%	72%
Naïve bayes	76%	66%	63%	66%	73%
Random forest	76%	68%	63%	66%	76%
XGboost	70%	58%	65%	61%	74%

(نتیجه گیری)

مواردی که accuracy بالایی دارند ولی با دقت precision و recall آنها اختلاف زیادی دارد در مسایل حساس هدف ما نیستند و مناسب نیستند از طرفی هم با بدست آوردن k-fold accuracy میتوانیم به دقت واقع بینانه تری از مدل ها دست یابیم. (random forest واقع بینانه ترین accuracy را در تلاش اول برای محاسبه accuracy بدست آورده است و باقی مدل ها دارای افت چند در صدی در محاسبه دوم accuracy که با روش 5-fold بود داشته اند.) همچنین f1-measure نیز در مقایسه عملکرد مدل ها برایمان مهم است چراکه نشان دهنده آن است که یک تعادل و trade off بین دو مقدار recall و precision است.

اگر بخواهیم 3 مدل برتر را انتخاب کنیم:

C4.5 به طور کلی بهترین مدل است زیرا:

1. دارای بالاترین recall و f1-measure است که نشان دهنده تعمیم پذیری (generalization) و تعادل و بالانس خوبی بین recall و f1-measure دارد.
 2. همچنین دقت آن فقط 1٪ کمتر از CART است، اما در recall عملکرد خوبی را دارد، که برای مسایلی که شناسایی موارد مثبت مهمتر است، بسیار مهم است و برایمان دارای اهمیت بالاتری است.
- CART همچنین از نظر accuracy و precision عملکرد خوبی را نشان داده است و آن را برای مسایلی که بر کاهش مثبت کاذب (FP) متمرکز شده اند مناسب می کند.

Random Forest بهترین 5-fold accuracy را به دست آورده است، که نشان می دهد برای مسایلی که در آنها robustness برایمان مهم است، این مدل مناسب تر است و کمتر مستعد overfeeding است.

اگرچه باید به این نکته اشاره کرد که در صورتیکه دو درخت cart و c4.5 را محدودتر کنیم، دقت بدست آمده برای مدل naïve bayes از آنها پیشی خواهد گرفت چراکه با گسترش بیشتر درخت مدل بهتر و دقیق تر آموزش می بیند اما همچنین گسترش دادن تمام و کمال درخت توصیه نمیشود چراکه میتواند موجب overfeet شدن مدل شود. (نتایج دقت بدست آمده برای درخت هایی با عمق 3 برای دو مدل cart و c4.5)

CART Model Accuracy: 0.7597402597402597

C4.5 Model Accuracy: 0.7662337662337663

Naive Bayes Model Accuracy: 0.7662337662337663

هر یک از معیارهای ارزیابی عملکرد مدل های هوش مصنوعی چه زمانی به کار میروند؟
اگر ماتریس confusion زیر را داشته باشیم تمامی معیارها را محاسبه کنید.

Predict → actual ↓	P	N
P	603	72
N	120	209

<< معیار های ارزیابی و کاربرد آنها به شرح زیر میباشند.

• Accuracy

زمانی که می‌خواهیم دید کلی و ساده‌ای از عملکرد مدل داشته باشیم و مجموعه داده متعادل (Balanced Data set) داریم، از این معیار استفاده می‌کنیم همچنین در مسائلی که عواقب و هزینه تشخیص های False Positive و False Negative برایمان به یک اندازه می‌باشد. (در مسائلی که تمامی پیش بینی های نادرست اثرات یکسانی دارند.) از این معیار نمیتوان برای Imbalanced Data set ها استفاده کرد.

برای مثال در تشخیص میزان وفاداری مشتریان

$$\frac{TP + TN}{ALL}$$

• Precision

نشان می‌دهد که، از بین تمام پیش‌بینی‌های مثبت، چند مورد واقعاً صحیح بودند؟

زمانی که موارد False Positive برایمان غیرقابل قبول هستند و اثرات و هزینه سنگینی برا برایمان به همراه دارند از این معیار استفاده می‌کنیم همچنین در مسائلی که می‌خواهیم خطر تشخیص های False Positive را به حداقل برسانیم.

برای مثال در تشخیص ایمیل های spam چراکه تشخیص یک ایمیل کاری مهم به عنوان spam نتیجه نامطلوبی را به همراه دارد.

$$\frac{TP}{TP + FP}$$

• Recall

نشان می‌دهد که، از بین تمام موارد مثبت واقعی، چند مورد به درستی شناسایی شدند؟

زمانیکه موارد False Negative تشخیص داده شده هزینه بر و دارای عواقب سنگینی برایمان است از این معیار استفاده میکنیم همچنین مسائلی که از دست دادن موارد مثبت واقعی پیامدهای شدیدی دارد.

برای مثال در تشخیص های پزشکی (از دست دادن یک مورد بیماری حیاتی است) یا در تشخیص تقلب های بانکی (از دست دادن یک مورد تقلب هزینه بر است).

$$TP / TP + FN$$

• F-measure

زمانیکه قصد داریم تعادلی را بین Accuracy و Precision حفظ کنیم یعنی هر دو این مقادیر را در نظر بگیریم از این معیار استفاده میکنیم همچنین در مسائلی که هم مثبت کاذب و هم منفی کاذب هزینه بر هستند و نیاز به حفظ تعادل خاصی بین این دو داریم.

$$2 * PRECISION * RECALL / PRECISION + RECALL$$

• F-Beta score

کاربردی همانند معیار قبلی دارد اما با این تفاوت که میتوانیم با مشخص کردن معیار beta میزان اهمیت و تاثیر هر یک از دو معیار Precision و Accuracy را کمتر یا بیشتر کنیم و در نظر بگیریم.

$$(1 + \beta^2) * PRECISION * RECALL / (\beta^2 * PRECISION * RECALL)$$

مقادیر معیارهای نام برده شده به ازای ماتریس confusion بالا به شرح زیر است:

$$0.812 = \frac{812}{1000} : \text{Accuracy}$$

$$0.83 = \frac{603}{723} : \text{Precision}$$

$$0.89 = \frac{603}{675} : \text{Recall}$$

$$2.67 = \frac{2 * 0.83 * 0.89}{0.83 + 0.89} : \text{F1-measure}$$

:F-Beta score

$$0.682 = (1 + 2^2) * 0.83 * 0.89 / (2^2) * 0.83 * 0.89$$

برای معیار F-Beta score، مقدار Beta را 2 در نظر گرفته ام. (به ازای $\beta > 1$ ، تمرکز بیشتری بر روی مقداری precision است و به ازای $\beta > 1$ ، تمرکز بیشتری بر روی مقدار Recall است).